Search + CSP + Neural Network Student Projects

Required deliverables for EVERY project (common checklist)

1. Source code in clearly named modules (data, nn, search, csp, integration, eval, tests).
2. Data script (make_data.py) that downloads and preprocesses the dataset reproducibly (fixed seed).
3. Data dictionary: every column, whether real or generated, and how generated parts were made.
4. Trained NN (or training script) with test results compared against at least one baseline.
5. Search implementation with counters (nodes expanded/generated), tested on a small example with a known answer.
6. CSP implementation: Backtracking, Forward Checking and Arc Consistency (AC-3) as separate switchable parts, with counters (backtracks, assignments).
7. One integrated demo: python run_demo.py shows the input, the solution, and plots of how the three parts cooperate.
8. README with environment setup (pinned requirements) and exact run commands.
9. Tests: at least 5 normal, 3 edge, 2 impossible cases (impossible cases must give a clear "no solution" message).
10. Experiments: tables and plots for the project's three experiments, at least 5 runs/seeds, mean and standard deviation.
11. Report (max 12 pages): problem, design choices, NN, integration, limits, conclusions.
12. 10-minute presentation plus 5-minute live demo showing the search trace, CSP pruning with and without FC/AC-3, and the system with NN on versus off.
13. Individual contribution statement (max 200 words each) plus the git log.

P1. Assessment-Aware Study Planner

In simple terms: A planner for a distance-learning student. It decides how many study sessions to give each upcoming assignment, and in which weeks, so the predicted marks reach a target with the fewest sessions.

Example: Target 55 on three assignments, 3 free study slots per week. Output: 4 sessions for assignment 1, 5 for assignment 2, 3 for assignment 3, plus a week-by-week timetable and predicted marks.

Real-life users: Students and academic advisers (decision support only).

The full idea

Neural network does: Predicts the mark on an assignment from earlier marks and recent online-learning activity (clicks).

Search does: Starts with zero sessions and adds one session at a time to one assignment. Uses Uniform Cost Search and Greedy Best-First (Extension: A*) to find the smallest plan whose predicted marks meet the target.

CSP does: Variables are the study sessions; values are weekly time slots. Rules: a session must come before its assignment deadline, one session per slot, sessions for the same assignment are in order. Backtracking places them, Forward Checking removes used slots, AC-3 trims impossible slots first.

How they work together: Search proposes a plan, the NN says whether marks are high enough, then the CSP checks the sessions can really be scheduled. If the CSP fails, that plan is discarded and Search continues.

If no solution exists: Report that the target is unreachable at this capacity and show the best predicted marks that are possible.

Dataset

Link: https://analyse.kmi.open.ac.uk/open_dataset

Extra link: https://figshare.com/articles/OULAD_Open_University_Learning_Analytics_Dataset/5081998

Name: Open University Learning Analytics Dataset (OULAD)

What it contains: 22 course presentations, 32,593 students, assessment results, and daily summaries of student clicks in the online learning environment (10,655,280 entries). CSV tables linked by identifiers.

Inputs and target for the NN: Inputs (about 12): clicks in the 4 weeks before the deadline split into 3 activity groups, previous scores, assessment type and weight, weeks since start. Target: the assessment score (0-100). Do NOT use gender, age, region, disability or deprivation columns.

Real vs generated: Scores and clicks are real. The "session length in clicks", slot capacity and target mark are generated by a documented rule.

Access, licence, size: Public, licence CC BY 4.0, figshare copy is 44.58 MB (CSV). Confirm no registration is needed.

Preprocessing and subset: Pick one course; filter the large click table in chunks and aggregate to weekly totals; train on earlier presentations and test on a later one.

Warnings: Confirm exact table and column names in Week 1. The planner is a "what-if" tool: link between

effort and marks is observational, not proof of cause. The report must say this.

Project-specific deliverables (in addition to the common checklist)

- Planner that outputs allocation, timetable and predicted marks for at least 30 student profiles.
- Check that predicted marks do not go DOWN when study effort goes up (monotonicity test) and report violations.
- Written limitations section on cause versus correlation.
- Demo of a case where the NN says "enough" but the CSP rejects the schedule.

Minimum version: One course, 3 assignments, UCS + Greedy, BT + FC + AC-3, small neural network.

Optional extension: A* with a monotone heuristic, maintaining arc consistency during search, per-student history.

The three experiments: (1) UCS vs Greedy vs A*: nodes expanded and cost. (2) BT vs BT+FC vs BT+FC+AC-3 with 2, 3, 4 slots per week. (3) NN versus a simple average-mark predictor inside the planner.

P2. Home Energy Planner

In simple terms: Plans a household's next day of electricity use. It guesses how much power the house uses anyway, chooses start times for flexible appliances, and plans when to charge or use a home battery to pay the least.

Example: Forecast says 3.4 kW at 18:00 and the limit is 5 kW. A 1.8 kW dishwasher at 18:00 would break the limit, so it is moved to 21:00. The battery then covers expensive evening hours.

Real-life users: Households and energy advisers (simulation only).

The full idea

Neural network does: Forecasts the next 24 hourly electricity loads from the previous day, same hour last week, hour, weekday and month.

Search does: Plans the battery hour by hour. State = (hour, battery level). Actions: charge, discharge, idle. Cost = price times electricity bought from the grid. Uses Uniform Cost Search and A* (heuristic: remaining energy not covered by the battery times the cheapest price).

CSP does: Variables are appliance start hours (washer, dryer, dishwasher, EV charger, oven). Rules: dryer after washer, no overlap on shared circuits, total load (forecast + appliances) under the limit each hour. Forward Checking removes start hours that would break the limit; AC-3 handles ordering and overlap rules.

How they work together: NN forecast -> CSP schedules appliances -> Search plans the battery. If the battery plan cannot stay under the limit, the clashing appliance combination is forbidden and the CSP runs again (up to 5 loops).

If no solution exists: List the appliance windows that cannot fit, offer to relax the limit in 10% steps and show the violation.

Dataset

Link: https://archive.ics.uci.edu/dataset/235/individual+household+electric+power+consumption

Name: Individual Household Electric Power Consumption (UCI)

What it contains: 2,075,259 one-minute measurements from one house near Paris, Dec 2006 to Nov 2010, 9 features (active and reactive power, voltage, current, three sub-meters). About 1.25% of rows have missing values.

Inputs and target for the NN: Inputs: previous 24 hourly averages, same hour last week, time features. Target: next-day hourly load (24 values).

Real vs generated: Electricity use is real. Prices, appliance power profiles, battery size and the power limit are generated (config file). Savings are simulated, not measured.

Access, licence, size: Public, licence CC BY 4.0. Archive size not verified.

Preprocessing and subset: Aggregate minutes to hours, fill gaps up to 3 hours, drop worse days. Chronological split: early years train, last 3 months of training for validation, final year test. About 1,000 days are enough.

Warnings: One household only; no holidays or price data.

Project-specific deliverables (in addition to the common checklist)

- Forecasting model with test error versus "same hour yesterday" and "same hour last week".
- Appliance CSP with unit tests; battery search checked against a simple dynamic-programming answer.
- Evidence that the feedback loop (Search failure -> new CSP constraint) was triggered at least once.
- Every cost labelled as simulated.

Minimum version: Forecast + appliance CSP + UCS battery plan.

Optional extension: A* heuristic, feedback loop, battery efficiency losses.

The three experiments: (1) UCS vs A* nodes and time over 30 days, checking the heuristic never overestimates. (2) BT vs BT+FC vs BT+FC+AC-3 with 3, 5, 8 appliances. (3) NN forecast vs yesterday-baseline: cost and limit violations using the real load.

P3. Sprint Planner with Learned Effort Estimates

In simple terms: Reads software issues (title and description), predicts how many story points each needs, then plans sprints: which developer does which issue in which sprint, using as few sprints as possible.

Example: A backlog of 120 issues and 4 developers becomes a sprint table with predicted points per developer and a report of how overloaded each sprint really was.

Real-life users: Agile software teams.

The full idea

Neural network does: Predicts story points from the issue text (word-importance features fed to a small network).

Search does: State = (sprint number, remaining issues). Each action commits one of up to 3 sprint plans produced by the CSP. Cost = 1 per sprint. A* with heuristic = ceiling of remaining predicted points divided by sprint capacity (never overestimates). Greedy and UCS are baselines.

CSP does: For one sprint: variable = each ready issue, value = a developer or "defer". Rules: developer has the right skill, linked issues go to the same developer, two risky issues not to the same developer, developer capacity not exceeded, sprint at least 90% full. BT + FC + AC-3.

How they work together: NN gives points -> Search picks the next sprint state -> CSP builds sprint plans -> A* keeps the cheapest sequence. Real points are used only at the end to measure overload.

If no solution exists: Issues that no developer can take are reported as unschedulable; the rest are planned.

Dataset

Link: https://data.mendeley.com/datasets/skk2wn9j86/1

Name: User Story NeoDataset (Mendeley Data), DOI 10.17632/skk2wn9j86.1

What it contains: 20,479 user stories (issues) from 33 GitLab projects with titles, descriptions and story points (12,262.7 points in total); only closed issues with a points value; collected Jan-Apr 2023.

Inputs and target for the NN: Input: title + description text. Target: story points (use log(1+points)).

Real vs generated: Text and points are real. Developer skills, capacities and issue dependencies are NOT in the data: students generate them with a seeded rule and must repeat experiments with 3 seeds.

Access, licence, size: Public on Mendeley. Licence not seen: check before sharing. The page mentions a Hugging Face copy (hf://datasets/giseldo/neodataset/issues.csv); confirm columns and size.

Preprocessing and subset: Remove duplicates and rows without points; cap extreme values; train on about 25 projects and plan on 3 unseen projects with 150+ issues.

Warnings: Story points are noisy and differ between teams. The estimator may barely beat a median guess: report it honestly.

Project-specific deliverables (in addition to the common checklist)

- Estimator tested on unseen projects against median baselines.

- Seeded generator code and a sensitivity table for 3 seeds.
- Search checked on a 10-issue backlog against brute force.
- Overload analysis using real points.

Minimum version: NN + sprint CSP + A* over sprints.

Optional extension: Re-plan after each sprint when real points are revealed.

The three experiments: (1) A* vs Greedy vs UCS: sprints and nodes. (2) BT vs BT+FC vs BT+FC+AC-3 at 50, 100, 150 issues. (3) NN vs median predictor: overload ratio.

P4. Delivery Routing with Learned Travel Times

In simple terms: A dispatcher has about 20 deliveries with time windows and 3 vans. The system learns how long trips take in a real city, divides deliveries among the vans, and finds the best stop order for each van.

Example: Orders 7 and 12 have windows 10:00-10:30 and 10:10-10:40. The model predicts 38 minutes between them, so they cannot share a van.

Real-life users: Courier and logistics dispatchers.

The full idea

Neural network does: Predicts trip duration between city zones from pickup zone, drop-off zone, hour, weekday and typical distance.

Search does: For each van: state = (current stop, stops visited, clock time). Action = go to the next unvisited stop. Cost = predicted travel time plus waiting. A* with heuristic = sum of the cheapest predicted arrival time into each unvisited stop; UCS and Greedy as baselines. Up to 10 stops per van.

CSP does: Variable = van for each order (3 values). Rules: the window must be reachable from the depot; two orders that cannot meet both windows in either order must use different vans (rule created from NN predictions); van capacity. Backtracking, Forward Checking, AC-3 on the "different van" rules.

How they work together: NN time matrix -> CSP assigns orders to vans -> Search orders stops per van -> if a route is late, the first late pair becomes a new "different van" rule and the CSP re-runs (up to 5 loops).

If no solution exists: Report unserviceable orders with reasons; offer widening windows in 15-minute steps.

Dataset

Link: https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page

Name: NYC Taxi & Limousine Commission (TLC) Trip Record Data

What it contains: Yellow and green taxi trips with pickup/drop-off times and locations, trip distance, fares, payment type, passengers. Monthly Parquet files; taxi-zone lookup table and shapefile on the same page. TLC states it cannot guarantee accuracy.

Inputs and target for the NN: Inputs: pickup zone, drop-off zone, hour, weekday, zone-pair mean distance (from training data only). Target: trip duration in minutes = drop-off time minus pickup time (keep 1-120 min).

Real vs generated: Trips are real. Orders, windows, demands and van capacities are generated. Taxi times only approximate van times (optional documented factor 1.2).

Access, licence, size: Public, no registration seen. Example file pattern (from a third-party page, confirm on the TLC page): https://d37ci6vzurychx.cloudfront.net/trip-data/yellow_tripdata_2022-01.parquet. File size not verified.

Preprocessing and subset: One month, weekdays, the 20 busiest zones, about 300,000 trips; split by day (3 weeks train, last week test).

Warnings: Confirm zone-ID columns exist in the chosen month. Rare zone pairs at night; outliers.

Project-specific deliverables (in addition to the common checklist)

- Predicted zone-to-zone time matrix by hour.
- Order generator (seeded) and instance set.
- A* checked against brute force on 6 stops.
- Simulation that replays held-out real trip durations and counts late deliveries.

Minimum version: NN + assignment CSP + A* per van.

Optional extension: Return trip and service-time variation.

The three experiments: (1) A* vs UCS vs Greedy for 6-10 stops. (2) BT vs BT+FC vs BT+FC+AC-3 for 15, 20, 30 orders. (3) NN vs historical-average vs distance-based times: lateness in the replay.

P5. Network Lightpaths: Route and Wavelength Assignment

In simple terms: In an optical network each connection needs a route and a colour (wavelength). Two connections that share a cable must use different colours. The system forecasts the biggest traffic flows for next hour, then gives them routes and colours.

Example: The biggest predicted flow gets 3 candidate routes; AC-3 removes colours already used on shared cables; Forward Checking fixes route 2 with colour 2.

Real-life users: Network operators (simulation).

The full idea

Neural network does: Forecasts next-hour traffic for every sender-receiver pair from the last 24 hours, same hour last week and time features.

Search does: Finds candidate routes for each request. State = current node (with path so far). Cost = link length (or hops). UCS, A* (straight-line distance heuristic) and BFS (fewest hops). Returns up to 3 loop-free routes per request.

CSP does: Variable = each request; value = (route, wavelength). Rules: two requests sharing a cable need different wavelengths; route within reach limit. Backtracking serves highest predicted demand first; Forward Checking removes clashing (route, wavelength) pairs; AC-3 removes pairs with no compatible partner.

How they work together: NN picks the top K flows -> Search gives routes -> CSP assigns wavelengths -> if a request has no option, Search is re-run with more routes; if still none, the lowest-priority request is dropped (blocked).

If no solution exists: List blocked requests with reasons (no route within reach, wavelengths used up).

Dataset

Link: https://www.cs.utexas.edu/~yzhang/research/AbileneTM/readme.txt

Extra link: https://sndlib.put.poznan.pl/networks.overview.action

Name: Abilene traffic matrices (Yin Zhang) + SNDlib network topologies

What it contains: Real measurements of traffic between 12 routers every 5 minutes (one week per file, 2,016 matrices of 12x12 values) over about six months in 2004. SNDlib gives the Abilene graph (12 nodes, 15 links, 132 demands) and larger networks such as germany50 (50 nodes).

Inputs and target for the NN: Inputs per sender-receiver pair: previous 24 hourly demands, same hour last week, hour, weekday. Target: next-hour demand.

Real vs generated: Traffic is real. Number of wavelengths, lightpath capacity and reach limit are generated parameters.

Access, licence, size: Readme is public. A copy on IEEE DataPort is 159.20 MB but access terms (login) are not verified; I did not open the download links. Whether SNDlib has dynamic traffic for Abilene is not verified.

Preprocessing and subset: Match router order in the traffic files to SNDlib node names (Week-1 task; otherwise use the topology implied by the routing matrix). Aggregate to hourly; use 4 weeks (3 train, 1 test).

Warnings: Data from 2004; node mapping may be tricky; instructor should mirror the files.

Project-specific deliverables (in addition to the common checklist)

- Working node mapping and hourly dataset.
- Forecast with top-K hit rate versus last-hour baseline.
- Route search tested against known shortest paths.
- Heatmap of wavelength use and a list of blocked requests.

Minimum version: Forecast + UCS routes + wavelength CSP.

Optional extension: A* with distance heuristic on larger networks, priority-based blocking.

The three experiments: (1) BFS vs UCS vs A* on Abilene and a 50-node network. (2) BT vs BT+FC vs BT+FC+AC-3 for K=10, 20, 40 and 4 or 8 wavelengths. (3) NN vs last-hour baseline: share of true top-K traffic served and blocking.

P6. Plausible Tour Itineraries

In simple terms: A tourist gives a start place, an end place, a time budget and a must-see type of place. The system builds a sightseeing route that real travellers would plausibly follow and that fits the time.

Example: Edinburgh, 5 hours, must include a museum: Park -> Museum -> Castle -> Cafe -> Museum -> Park, with arrival times and visit durations.

Real-life users: Tourists and tour planners.

The full idea

Neural network does: (1) Predicts which place people visit next, given the current place and hour. (2) Predicts how long a visit lasts.

Search does: Builds the actual itinerary. State = (position in the plan, current place, visited set, elapsed time). Action = go to an unvisited place of the category required in this position. Cost = minus log of the NN transition probability plus travel time. UCS, A* and Greedy.

CSP does: Chooses the category pattern first. Variables = category of each stop; rules: neighbouring stops differ, the must-see category appears, no category more than twice, expected total time within budget. BT + FC + AC-3.

How they work together: CSP produces up to 10 valid category patterns -> Search finds the most plausible real places for each -> predicted durations are added; if the budget is exceeded the pattern is rejected -> the best itinerary is returned.

If no solution exists: Tell the user the smallest budget that works; if all patterns exceed the budget, return the cheapest itinerary with a warning.

Dataset

Link: https://github.com/computationalmedia/tour-cikm16

Name: Flickr tour datasets (Lim et al.), repository tour-cikm16

What it contains: Places of interest (id, category, longitude, latitude) and photo-based visit trajectories (place, start time, end time, number of photos, visit duration in seconds). Edinburgh, Melbourne and Toronto trajectories; Glasgow and Osaka place files also listed. Built from geotagged Flickr photos.

Inputs and target for the NN: Next-place model: current place, hour. Duration model: place, hour. Targets: next visited place; log duration.

Real vs generated: Trajectories are real. Walking speed, number of stops and budgets are generated.

Access, licence, size: Public GitHub repository; licence and file sizes not verified (small CSV files expected). Confirm user/trajectory id columns.

Preprocessing and subset: One city; keep places with at least 20 visits (at most 60 places); clip durations below 5 minutes; split by user if possible.

Warnings: Durations come from photo timestamps, so many are near zero and noisy.

Project-specific deliverables (in addition to the common checklist)

- Two trained NN heads with accuracy/NLL versus a count-based Markov baseline.
- Category-pattern CSP with unit tests.
- Search checked against brute force on a small pool.
- Comparison of produced itineraries with held-out real trajectories (F1).

Minimum version: Template CSP + UCS using NN costs + duration check.

Optional extension: A* tuning, a second city.

The three experiments: (1) UCS vs A* vs Greedy for 4-8 stops. (2) BT vs BT+FC vs BT+FC+AC-3. (3) NN costs vs Markov-count vs popularity: F1 against real trajectories.

P7. Handwritten Sudoku Reader and Solver

In simple terms: A computer reads a Sudoku written in handwritten digits. Sometimes it misreads a digit, which breaks the puzzle. The system finds the most likely misread digits, corrects them, and solves the puzzle.

Example: The network reads a 7 as a 1 (52% vs 45%). Two 1s appear in the same row, so the puzzle looks broken. The system changes that cell to 7 (cheap, since the network was nearly as sure about 7) and the puzzle becomes solvable.

Real-life users: Puzzle apps, education. Recommended project (cheapest compute, clear right answers).

The full idea

Neural network does: Classifies each handwritten digit image (1-9) and gives probabilities. Those probabilities become the cost of changing a cell.

Search does: State = set of corrections made to the first reading. Action = change one uncertain cell to one of its top alternatives. Cost = how much less likely the new digit is (never negative). UCS finds the most likely correction set; Greedy and A* are compared (A* heuristic: at least one cell of any duplicate-digit conflict must change). Maximum 3 corrections.

CSP does: 81 cells, values 1-9, givens fixed. Rule: cells in the same row, column or box differ. Backtracking solves; Forward Checking removes used digits from neighbours; AC-3 detects conflicts (two equal givens in one row empty a domain) and shrinks domains first.

How they work together: NN reads the grid -> AC-3 finds a conflict -> Search tries the cheapest corrections -> CSP tests each corrected puzzle and the first solvable one wins (Extension: require exactly one solution).

If no solution exists: Report "unreadable" and list the least confident cells.

Dataset

Link: http://yann.lecun.com/exdb/mnist/

Name: MNIST database of handwritten digits

What it contains: 60,000 training and 10,000 test images, 28x28 pixels, centred; training and test writers are different people. Four gzip files (about 11 MiB total).

Inputs and target for the NN: Input: 28x28 image. Target: digit 1-9 (drop digit 0). Blank cells are detected by a simple brightness rule, not by the NN.

Real vs generated: Digit images and labels are real. Sudoku puzzles are generated by students (build a full grid by randomised backtracking, remove cells while the solution stays unique); the 9x9 image mosaics and the injected hard digits are generated. This is not photographed real-world Sudoku.

Access, licence, size: Public. I did not open the original page directly (only pages that cite it); confirm it and keep a local copy. Library loaders usually exist.

Preprocessing and subset: Scale pixels to 0-1; split 50,000 train / 10,000 validation from the training set; test set untouched.

Warnings: If the network is too accurate there is nothing to correct: inject hard (low-confidence) test digits

using a documented rule. Give each team its own generator seed.

Project-specific deliverables (in addition to the common checklist)

- Calibrated classifier (accuracy, confusion matrix, calibration error) versus logistic regression.
- Puzzle and mosaic generator with seed.
- Correction log for each solved puzzle (cell, read digit, new digit, cost).
- Exact-solution recovery rate on 100 puzzles per misread level.

Minimum version: NN + full CSP + UCS with "puzzle solvable" goal, up to 2 injected misreads.

Optional extension: Uniqueness goal, A* heuristic, temperature scaling of probabilities.

The three experiments: (1) UCS vs Greedy vs A*: nodes and recovery rate for 0-4 misreads. (2) BT vs BT+FC vs BT+FC+AC-3 on puzzles with 25, 30, 35 givens. (3) NN-guided correction vs no correction vs random correction.

P8. Warehouse Re-Slotting from Predicted Demand

In simple terms: Decides which product goes on which shelf so pickers walk less next week. First it places products using predicted demand and "bought together" rules, then it improves the layout with a few swaps.

Example: A product predicted to sell 900 units is placed nearest the depot, and its most common partner product must be within a set distance of it.

Real-life users: Warehouse planners (simulation).

The full idea

Neural network does: Forecasts next-week units sold per product from the last 8 weeks, week of year and price.

Search does: Improves the layout. State = a complete valid layout; action = swap two products (only among the 30 highest-demand products and slots near the depot); cost = 1 per swap; goal = demand-weighted walking distance below a target or 6 swaps. Greedy Best-First (main), bounded UCS (baseline), A* variant (not guaranteed optimal).

CSP does: Variable = each product; value = shelf slot. Rules: one product per slot, heavy items on low shelves, frequently co-bought pairs within a maximum distance, incompatible categories not in the same aisle. BT + FC + AC-3 (limited to the 30 strongest pairs).

How they work together: NN demand -> CSP builds the first layout (high demand nearest the depot) -> Search tries swaps -> CSP rechecks every swap -> final layout is tested on real held-out orders.

If no solution exists: Relax the distance rule step by step and report the loosest value used.

Dataset

Link: https://archive.ics.uci.edu/dataset/502/online+retail+ii

Name: Online Retail II (UCI)

What it contains: 1,067,371 transactions of a UK online retailer, 1 Dec 2009 to 9 Dec 2011: invoice number, stock code, quantity, date (plus description, unit price, customer id, country). Some values missing; invoices starting with "C" are cancellations.

Inputs and target for the NN: Inputs per product-week: last 8 weeks of log(1+units), week-of-year, mean price, invoice count. Target: next-week units. Evaluation orders are real invoices of the test weeks.

Real vs generated: Sales and orders are real. Warehouse grid, slot capacities and heavy/incompatible flags are generated; distances are simulated.

Access, licence, size: Public; licence CC BY 4.0 per UCI (one redistributing package notes non-commercial use, which coursework satisfies). File size not verified; loader available via ucimlrepo (id 502).

Preprocessing and subset: Top 120 products by volume; remove cancellations; chronological split (about 70 weeks train, 15 validation, last weeks test).

Warnings: Simple baselines (last week) are strong in retail; the NN may not win. Christmas seasonality affects the test period.

Project-specific deliverables (in addition to the common checklist)

- Demand forecast versus last-week and 4-week-average baselines.
- Co-purchase pair list from training weeks only.
- Layout generator, picker-distance simulator on real held-out orders.
- Four-layout comparison: random, last-week demand, NN demand, NN + swaps.

Minimum version: NN-ordered CSP layout + Greedy swap search.

Optional extension: A* variant, second warehouse shape.

The three experiments: (1) Greedy vs bounded UCS vs A*: final distance, swaps, nodes. (2) BT vs BT+FC vs BT+FC+AC-3 for 60, 90, 120 products. (3) Picker distance over 10 test weeks for the four layouts.